Quantstamp

Polymarket CTF Exchange V2

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Prediction Market
Timeline	2026-03-02 through 2026-03-11
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Diff/Fork information	The v1 of the CTF exchange can be found here: https://github.com/Polymarket/ctf-exchange
Source Code	• Polymarket/ctf-exchange-v2 #a9d2397
Auditors	• Tim Sigl Auditing Engineer • Paul Clemson Auditing Engineer

Documentation quality	Medium
Test quality	High
Total Findings	5 Fixed: 5
High severity findings	0
Medium severity findings	1 Fixed: 1
Low severity findings	3 Fixed: 3
Undetermined severity findings	0
Informational findings	1 Fixed: 1

Summary of Findings

The Polymarket CTF Exchange v2 is a Conditional Tokens Framework (CTF) trading protocol that enables operator-controlled order matching for binary outcome markets. The system is composed of three subsystems: an exchange that matches signed orders and settles trades, a set of adapter contracts that bridge between the exchange's collateral token and the CTF, and a collateral layer that wraps USDC and USDC.e into a unified PolymarketCollateralToken (PMCT). Off-chain infrastructure handles order collection, matching, and submission, while on-chain contracts enforce settlement correctness, fee validation, and access control.

The exchange is centered on the `CTFExchange` contract, which inherits from a chain of mixin contracts, the most important ones being: `Trading` for order matching and settlement, `Fees` for fee receiver management, and rate validation and `Signatures` for preapproved and regular signature verification across four signer types (`EOA` , `POLY_PROXY` , `POLY_GNOSIS_SAFE` and `POLY_1271`). The exchange supports three order matching flows:

- **COMPLEMENTARY** matches a BUY taker against SELL makers (or vice versa) on the same outcome token, settling with direct peer-to-peer transfers.
- **MINT** matches a BUY taker against BUY makers on COMPLEMENTARY outcomes (e.g. YES and NO), pooling collateral in the exchange and minting both outcome tokens for distribution.
- **MERGE** matches a SELL taker against SELL makers, collecting both outcome tokens and merging them back into collateral.

When a single `matchOrders()` call includes COMPLEMENTARY makers together with one or more non-COMPLEMENTARY makers, that mixed flow is supported. For example, a BUY YES taker can match SELL YES makers through the COMPLEMENTARY transfer path while also matching BUY NO makers through MINT in the same call. There is also an all COMPLEMENTARY shortcut when every maker is COMPLEMENTARY. A single call does not support matching unrelated orders such as MINT and MERGE together.

The permissioned and permissionless on-ramps are the two main entry points into the onboarding liquidity to the CTF exchange. Together they control which and how many stablecoins users and partners can wrap into PMCT and unwrap again into USDC and USDC.e, with permissionless flows and a permissioned path for balancing the underlying stablecoin supply. PMCT itself serves as the shared collateral token across the system, abstracting over USDC and USDC.e so the exchange and its surrounding infrastructure can operate on a single asset.

The adapter layer sits between the exchange and the CTF to make CTF settlement compatible with PMCT. It converts between PMCT and the USDC.e collateral required by the CTF and supports both regular CTF markets and negative risk markets (multi-outcome) through separate adapters.

Compared with CTF Exchange v1, v2 adds preapproval, delayed user pause, builder/metadata fields, and PMCT adapter-based collateral flow, while removing nonce-based cancellation and older fill-style execution paths. These updates improve flexibility and gas efficiency, but also increase reliance on operator and off-chain matching logic, and led to regressions identified in this report, including removed complement-market validation in non-complementary matching (POL-EX-4) and unchecked arithmetic overflow behavior in settlement math (POL-EX-5).

The audit found no high severity issues, with one medium severity finding three low severity findings and one informational severity finding identified. The medium severity finding, POL-EX-1, is that inconsistent supplied fill inputs in the COMPLEMENTARY settlement path can overcharge or underfill takers because status updates are not tied to actual executed transfers.

The test suite comprises 227 tests across 23 test suites. It includes integration tests that exercise full `matchOrders()` flows with balance assertions, adapter-specific matching tests, and gas snapshot benchmarks across all flow types and different maker counts.

Fix-Review Update 2026-03-25:
During the fix review all the findings highlighted during the audit were fixed by the client. Additionally, the client implemented all of the auditor suggestions.

ID	DESCRIPTION	SEVERITY	STATUS
POL-EX-1	**Takers Can Be Overcharged or Underfilled in COMPLEMENTARY path**	Medium	Fixed
POL-EX-2	**Builder Volume Can Be Misattributed**	Low	Fixed
POL-EX-3	**Orders Where `cashValue == 0` Can Be Charged Uncapped Fees**	Low	Fixed
POL-EX-4	**Operator Can Route Value Across Non-COMPLEMENTARY Matches and Drain Stranded Inventory**	Low	Fixed
POL-EX-5	**Overflow in `calculateTakingAmount()` Can Produce Incorrect Settlement Amounts**	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:

1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Polymarket/ctf-exchange-v2`

Included Paths: `src`

Operational Considerations

- **The exchange uses an operator-centric trust model:** operators control `matchOrders()` inputs (`conditionId` , `takerFillAmount` , `makerFillAmounts` , fees) and also control preapproval/invalidation flows. Therefore, correct user order execution depends heavily on correct and honest off-chain operator logic. If operator logic is buggy or compromised, outcomes can include value misrouting and stranded inventory, incorrect fill/accounting emissions, and extreme-parameter mis-settlement. Users should treat this as a high-trust operational assumption, not a low-trust on-chain guarantee.
- **Order cancellation relies on off-chain logic:** The current order structure is the result of a design revision: Nonces were removed because they previously acted as a global order cancellation mechanism but were removed after being identified as a griefing vector that allowed unexpected invalidation of orders currently in matching. As a consequence, there is no on-chain way for a maker to cancel a single order or to have orders expire automatically. Users should be aware that order cancellation relies on off-chain logic and on `pauseUser()` with its delay window, rather than on per-order cancel or expiration.
- `CollateralToken` **vault must hold sufficient reserves in USDC.e:** In the current implementation of the `CollateralToken` contract, both Polygon native USDC and bridged USDC.e can be wrapped to mint collateral tokens. However, when collateral tokens are used to interact with conditional tokens via the `CtfCollateralAdapter` contract, it is necessary that the collateral token being used is first unwrapped to USDC.e. Therefore, Polymarket's on-ramp providers must ensure a sufficient amount of USDC.e is always present in the collateral token vault else risk interactions with conditional tokens failing.
- `MINTER_ROLE` **in** `CollateralToken` **can mint unbacked collateral tokens:** It is important within the protocol that the `CollateralToken` remains backed 1:1 (by a combination of USDC and USDC.e). However, an address with the `MINTER_ROLE` is able to call `mint()` to mint arbitrary amounts of unbacked collateral tokens. This role is expected to be unassigned initially in production and if used in the future it will be assigned to a smart contract, which would be trusted to handle the responsibility correctly.
- **Stale contract state during ERC-1155 callbacks:** Multiple external calls occur during `matchOrders()` that trigger ERC-1155 callbacks on recipient contracts. Between those calls, any reader can observe state that is not yet in its final post-settlement form. One example: the exchange's collateral and CTF token balances are partially updated between individual transfers inside `_settleComplementaryMaker()` and `_settleMakerOrders()`, so a contract that checks the exchange's `balanceOf()` on token reception may see values that do not reflect the final settlement. The exchange currently has no function that returns stale data or operates on the stale state during token reception.
- **Deterministic PolySafe Wallet Address** When signing Polymarket orders for a Polymarket Gnosis Safe wallet, users will be unable to change the signer address (even if they are able to change the safe's owner), as the original signer address is used to derive the wallet address during signature validation.

Key Actors And Their Capabilities

Exchange

Exchange Operator

Responsibilities

- Call `CTFExchange.matchOrders()` to match a taker order against a list of maker orders.
- Charges fees to be paid by each matched order.
- Can preapprove user order signatures via the `preapproveOrder()` function.
- Can renounce their own operator role via the `renounceOperatorRole()` function.

Trust Assumptions

- The operator is trusted to process user orders and not unfairly censor user's ability to interact with the protocol.
- The operator is trusted to charge users reasonable fees for fulfilling their orders.
- The operator is trusted to match orders efficiently, and not extract any surplus value that may be present by cherry picking matches.
- The operator is trusted to drop unfilled orders after some reasonable amount of time, avoiding scenarios where stale user orders are filled long after the order was originally created.

Exchange Admin

Responsibilities

- Pause/unpause order matching.
- Set the `userPauseBlockInterval`, a number of blocks after which a user can pause themselves.
- Set the fee receiver for trading fees.

- Set the `maxFeeRateBps` the maximum percentage of fees operators are allowed to charge.
- Add and remove other protocol admins, or renounce their own admin role.
- Add and remove protocol operators.

Trust Assumptions

- The admin is trusted to set a reasonable `maxFeeRateBps`.
- The admin is trusted to ensure sufficient actors with the `admin` and `operator` roles are present to ensure the protocol continues to run smoothly.

Collateral Token

Polymarket `CollateralToken` Owner

Responsibilities

- Manage the `MINTER_ROLE` and `WRAPPER_ROLE` in `CollateralToken`.
- Authorize upgrades to `CollateralToken`.
- Can assume minter/wrapper capabilities by granting `MINTER_ROLE` and/or `WRAPPER_ROLE` to itself.

Trust Assumptions

- The `MINTER_ROLE` can arbitrarily mint unbacked collateral tokens, so access to this role should be handled with care.

Polymarket `CollateralToken` Minter

Responsibilities

- Mint `CollateralToken` to arbitrary addresses and burn PMCT from the minter address.

Trust Assumptions

- It is expected that this role is owned by a smart contract, that will ensure mints and burns only occur as necessary.
- Does not mint `CollateralToken` without backing stablecoins.

Polymarket `CollateralToken` Wrapper (Ramps)

Responsibilities

- Execute `wrap()` and `unwrap()` which gives operational control over movement between `CollateralToken` and the underlying stablecoins.

Trust Assumptions

- Only authorized ramps are assigned the wrapper role.
- Ensures one-sided unwrap operations cannot drain any of the underlying stablecoins.

Ramps

Ramp Owner

Responsibilities

- As owner of `CollateralOnramp`, `CollateralOfframp`, and `PermissionedRamp`, can ultimately reassign ramp roles through inherited ownership powers.
- Can assume ramp admin capabilities by granting `_ROLE_0` to itself.

Trust Assumptions

- Ramp owners are top level authorities over ramp administration and must not install malicious admins or witnesses.

Ramp Admin

Responsibilities

- Add or remove admins on the ramps.
- Pause or unpause asset specific ramp flows.
- In `PermissionedRamp`, also add or remove witnesses.

Trust Assumptions

- Ramp admins can censor onboarding or offboarding flows and control the witness set, so they must not abuse pause power or onboard untrusted witnesses.

Permissioned Ramp Witness

Responsibilities

- Approve permissioned wrap and unwrap requests by signing the EIP-712 witness payloads verified by `PermissionedRamp`.

Trust Assumptions

- Witness keys must remain secure and must only sign intended authorizations.

Findings

POL-EX-1

Takers Can Be Overcharged or Underfilled in COMPLEMENTARY path

Medium Fixed

Update

Fixed in commit `1af9851721052947c4ca0fe11189812083647968`.

File(s) affected: `src/exchange/mixins/Trading.sol`

Description: In the COMPLEMENTARY settlement path, the parameter `takerFillAmount` only updates the taker's `orderStatus.remaining` capacity but does not constrain the actual tokens flowing to or from the taker. The taker's real inflows and outflows are determined entirely by the makers' orders and the `makerFillAmounts[i]` supplied by the operator. Because `takerFillAmount` and `makerFillAmounts[i]` are independent inputs and are not in the same asset units in the COMPLEMENTARY path, there is no on-chain check that aggregate maker-side execution is economically consistent with the declared taker fill; as a result, settlements can violate the taker's signed constraints without being rejected. The missing enforcement opens two categories of incorrect settlement:

1. **Overspend.** If the off-chain logic submits a small `takerFillAmount` (consuming little order capacity) with disproportionately large `makerFillAmounts`, the taker's actual collateral (or CTF) outflow, computed from the makers' order ratios, can exceed the taker's signed `makerAmount`. Because each such match consumes only a sliver of order capacity, the same taker order can be matched repeatedly with new makers, each time pulling more funds than authorized, until the taker's balance or approval is exhausted.
2. **Underpay.** If the off-chain logic submits a large `takerFillAmount` (consuming most or all order capacity) with insufficient maker fills, the taker's order is fully consumed while the taker receives a fraction of the tokens their ratio guarantees. The `taking` value (the minimum output the taker's ratio entitles them to) is computed by `_performOrderChecks()` but is never checked or enforced in the COMPLEMENTARY branch. Compare this to the non-COMPLEMENTARY path, where `_updateTakingWithSurplus(taking, takerAssetId)` at line 112 enforces the minimum.

It is important to note that the MINT/MERGE path is not affected: there the taker transfers `takerFillAmount` to the exchange upfront, and `_updateTakingWithSurplus()` enforces the minimum output. In those paths, even a faulty off-chain submission cannot produce an out-of-bounds settlement.

Exploit Scenario:
Scenario 1 (Overspend):

1. Alice signs a BUY order: `makerAmount=100` USDC, `takerAmount=50` CTF (limit price: 2 USDC per CTF).
2. Bob creates a maker SELL order with `makerAmount=100` CTF, `takerAmount=150` USDC (price: 1.5 USDC per CTF). This crosses with Alice's order (the crossing check in `_validateOrdersMatch()` passes: `100*100 >= 50*150` i.e. 10000 >= 7500).
3. The operator calls `matchOrders()` with `takerFillAmount=100` and `makerFillAmounts=[100]` (one full fill).
4. The maker's `taking` is `(100 * 150) / 100 = 150` USDC (calculated in `_settleComplementaryMaker() -> _performOrderChecks() -> calculateTakingAmount()`).
5. Alice pays 150 USDC directly to the maker via `_settleComplementaryMaker() -> takerIsBuy branch -> _transfer(taker, makerOrder.maker, 0, 150)`.
6. Alice spent 150 USDC (exceeding her signed `makerAmount` of 100) for 100 CTF in a single execution.

Scenario 2 (Underpay):
1. Alice signs a BUY order: `makerAmount=100` USDC, `takerAmount=100` CTF (1:1 ratio).
2. The operator matches with `takerFillAmount=100` and a single maker who fills only 10 CTF.
3. `taking` is computed as 100 CTF but is never checked. Alice receives 10 CTF, pays based on the maker's price, and her order is fully consumed (`remaining=0`, `filled=true`).
4. Alice's order is gone and she received 10% of expected volume.

Recommendation: In the COMPLEMENTARY path, defer the taker's order status update to after the maker settlement loop and base it on actual settlement values:

1. Before the maker loop, validate the taker order but do not call `_updateOrderStatus()`.
2. During the maker loop, accumulate `totalTakerSpent` as `sum(taking_i)` across all makers, where each `taking_i` is the settlement obligation computed from the maker's ratio via `calculateTakingAmount()`. This represents the actual collateral spent (taker BUY) or CTF sent (taker SELL).
3. After the loop, call `_updateOrderStatus(takerOrderHash, takerOrder, totalTakerSpent)` so that `remaining` reflects actual consumption rather than the operator supplied `takerFillAmount`.

With this change, `takerFillAmount` is no longer needed in the COMPLEMENTARY branch. Note that deferring the status update means `remaining` is not yet updated and stale when makers and the taker receive conditional tokens which opens a reentrancy vector. Since `matchOrders()` is operator restricted, reentrancy through ERC-1155 callbacks is not a practical concern in order matching, but status values read during token reception via `getOrderStatus()` will be stale (read-only reentrancy). If protection from a reentrant operator is needed consider adding a reentrancy guard to the functions in question.

POL-EX-2 Builder Volume Can Be Misattributed

Low · Fixed

Update
Fixed in `fd2f1a99fb9c3897fa0341ed4d7358ee160a24d3`.

File(s) affected: `src/exchange/mixins/Trading.sol`

Description: The values emitted from the `OrderFilled` event are important for attributing correct builder rewards and fee sharing. The `builder` value itself is signed as part of the order payload and included in the EIP-712 struct hash. To ensure that these rewards are calculated correctly, it is important that the emitted volume value from the `OrderFilled` event match exactly the amounts executed in the trade.

In v2, emitted taker volume is not always derived from per-match transfer deltas. In COMPLEMENTARY settlement, `_settleComplementaryTaker()` finalizes taker `OrderFilled` values from `takerFillAmount`. But actual transfers are executed in `_settleComplementaryMaker()` from `makerFillAmounts`. In non-COMPLEMENTARY settlement, `_updateTakingWithSurplus` can replace expected taking with full exchange balance for that asset before taker event emission.

The following scenarios can result in incorrect `OrderFilled` volume:

1. Empty maker list in COMPLEMENTARY branch can still emit non-zero taker fill.
2. COMPLEMENTARY taker BUY can emit collateral-side input as `takerAmountFilled` instead of conditional tokens received.
3. COMPLEMENTARY `takerFillAmount = 0` with non-zero maker fills can emit zero taker fill while real transfers occur.
4. Non-COMPLEMENTARY taker fill can include pooled or pre-existing surplus, not only current-match execution.

If rewards are paid from raw emitted volume, this can over-credit, under-credit, or misattribute builder performance.

Recommendation:

1. `_isAllComplementary()` should revert if the length of the `makerOrders` array is zero.
2. Fix the COMPLEMENTARY taker BUY unit mismatch by emitting actual conditional tokens received instead of `takerFillAmount` (if POL-EX-1 is implemented `totalTakerSpent` can be used here)
3. Make executed transfer deltas the source of truth for taker status updates and taker `OrderFilled` fields, instead of using raw input values. (if POL-EX-1 is implemented `totalTakerSpent` can be used here)
4. For `OrderFilled` accounting, exclude pooled or pre-existing surplus from emitted fill amounts and emit only per-match executed deltas.

POL-EX-3

Orders Where `cashValue == 0` Can Be Charged Uncapped Fees

• Low ⓘ Fixed

Update

Fixed in commit `59368127e70009cb77fa5a205210bd2246d5505c`.

File(s) affected: `src/exchange/mixins/Fees.sol`

Description: The fee amounts charged on orders in the `CTFExchange` contract are dynamic and may be unknown to the user at the time the order is created. To limit the trust required of the operator, the total fee to be paid by an order is validated to be within some accepted `maxFeeRateBps` in the `validateFeeWithMaxFeeRate()` function. However, in scenarios where the `cashValue` of an order is zero, the `validateFeeWithMaxFeeRate()` function returns before these checks are made, meaning such orders can be charged an uncapped fee. As the `cashValue` is based on values supplied by the operator (`takerFillAmount` or `makerFillAmounts`) and not a value included in the order struct signed by the user, the user cannot reasonably prevent this.

Exploit Scenario:

Consider the following scenario:

```
function test_ZeroFillBuyOrder_FeeBypassDrainsCollateral() public {
    // maxFeeRateBps is the default 500 (5%).
    // Bob has 10,000 USDC approved — far more than his 100 USDC order.
    uint256 bobInitialBalance = 10_000_000_000; // 10,000 USDC (6 decimals)
    dealUsdcAndApprove(bob, address(exchange), bobInitialBalance);

    // Bob signs a BUY order: spend up to 100 USDC for 100 YES tokens
    Order memory takerOrder = _createAndSignOrder(bobPK, yes, 100_000_000, 100_000_000, Side.BUY);

    // Empty maker arrays — complementary path with zero actual matches
    Order[] memory makerOrders = new Order[](0);
    uint256[] memory makerFillAmounts = new uint256[](0);
    uint256[] memory makerFeeAmounts = new uint256[](0);

    // --- Attack round 1: drain 5,000 USDC as fee ---
    vm.prank(admin);
    exchange.matchOrders(
        conditionId, takerOrder, makerOrders, 0, makerFillAmounts, 5_000_000_000, makerFeeAmounts
    );

    // Bob lost 5,000 USDC despite the 5% fee cap (legitimate max fee would be 5 USDC)
    assertCollateralBalance(bob, 5_000_000_000);
    assertCollateralBalance(feeReceiver, 5_000_000_000);

    // Order is NOT consumed — remaining capacity unchanged
    assertFalse(exchange.getOrderStatus(exchange.hashOrder(takerOrder)).filled);

    // --- Attack round 2: drain remaining 5,000 USDC with the same unfilled order ---
    vm.prank(admin);
    exchange.matchOrders(
```

```
        conditionId, takerOrder, makerOrders, 0, makerFillAmounts, 5_000_000_000, makerFeeAmounts
    );

    // Bob fully drained
    assertCollateralBalance(bob, 0);
    assertCollateralBalance(feeReceiver, bobInitialBalance);

    // Order is STILL not consumed — attack can continue if Bob receives more USDC
    assertFalse(exchange.getOrderStatus(exchange.hashOrder(takerOrder)).filled);
}
```

Recommendation: Consider capping fee amount to some static amount when `cashValue == 0`. Alternatively, if there are no valid use cases for orders where `cashValue == 0` consider reverting when such orders are included in a `matchOrders()` transaction.

POL-EX-4

Operator Can Route Value Across Non-COMPLEMENTARY Matches and Drain Stranded Inventory

• Low Fixed

Update
Fixed in commits `38aac63094fc765ca62d66f61f4f4a89b74c4ee6` and `fd2f1a99fb9c3897fa0341ed4d7358ee160a24d3`.

File(s) affected: `src/exchange/mixins/Trading.sol`

Description: In non-COMPLEMENTARY matching (MINT/MERGE), settlement is pooled at the exchange level instead of isolated per match. In `_matchOrders()`, taker and maker assets move into shared exchange balances. `_settleMakerOrders()` then executes maker matches in batch, and `_settleTakerOrder()` pays the taker from the exchange inventory.

During the non-COMPLEMENTARY MINT/MERGE matching flow, `_mint()` / `_merge()` use the `conditionId` supplied to `matchOrders()`, while payout assets are derived from order `tokenId`s. Non-COMPLEMENTARY matches can produce surplus from price crossing: in MINT, surplus exists when taker price + maker price > 1; in MERGE, surplus exists when taker price + maker price < 1.

In v1, non-COMPLEMENTARY matches required the maker `tokenId` to be the complement of the taker `tokenId` via `validateComplement()`. In v2, the complement check was removed, allowing mixed maker composition on-chain.

The matching logic assumes that the maker `tokenId`s in a match correspond to outcomes of the provided `conditionId`, so that any non-COMPLEMENTARY surplus remains confined to the intended match. Because this relationship is not enforced on-chain in v2, mismatches can occur even without malicious intent (e.g., batching mistakes or matcher bugs).

When mixed batches do not align with the intended match composition, settlement can leave stranded inventory in the shared exchange balances. This breaks per-order value isolation because later matches can consume value created by earlier unrelated matches, even when each order still appears valid against its signed limit when checked individually.

`_updateTakingWithSurplus()` can increase the taker payout from the match minimum to the exchange's full balance of that taker asset. `_settleTakerOrder()` only refunds leftover taker `makerAssetId`. These functions together allow surplus and pooled balances to be routed across unrelated matches and across batches.

Variants:

1. Accidental / matcher-bug variant (more likely): Same-market mixed maker composition (e.g., batching or matching errors) can strand side-token balances even when `conditionId` is correct.
2. Trust-failure variant (less likely): cross-market mixed batches with operator-provided `conditionId` can mint or merge one market while paying another, also creating stranded inventory.

Operator extraction method:

1. After stranded inventory exists, submit a small follow-up order where taker asset equals the stranded asset.
2. `_updateTakingWithSurplus()` uses full exchange balance (not per-match delta), so payout can include that pre-existing stranded inventory.
3. `_settleTakerOrder()` only refunds taker `makerAssetId`, so other residual balances remain available for later extraction.

Exploit Scenario:
Scenario 1:

1. Taker: `BUY YES_A` (`maker=60`, `taker=100`), fill `60`.
2. Maker1: `BUY YES_A` (`maker=30`, `taker=50`), fill `30`.
3. Maker2: `BUY NO_A` (`maker=90`, `taker=100`), fill `90`.
4. Pull collateral: `60 + 30 + 90 = 180` into exchange.
5. Mint amount is `50 + 100 = 150`, so exchange mints `150 YES_A + 150 NO_A` and keeps `30` collateral.
6. Maker payouts: `50 YES_A` to Maker1 and `100 NO_A` to Maker2.
7. Taker payout/refund: `100 YES_A` plus `30` collateral refund.
8. Result: exchange still holds `50 NO_A` stranded, even though all orders are market `A`.
9. v1 contrast: this mixed composition would fail because non-COMPLEMENTARY matching required maker `tokenId` to be the complement of taker `tokenId`.

Scenario 2:

1. Pre-state: exchange holds stranded `50 NO_A` from Scenario 1.
2. Operator submits a small follow-up non-COMPLEMENTARY match where taker asset is `NO_A`.
3. `_updateTakingWithSurplus` returns full exchange `NO_A` balance, not only what this follow-up match produced.
4. Result: taker payout includes previously stranded `50 NO_A`, extracting value from prior unrelated settlement.

Scenario 3:

1. Taker: `BUY YES_A` (`maker=60`, `taker=100`).
2. Maker1: `BUY NO_B` (`maker=50`, `taker=100`).
3. Maker2: `SELL YES_A` (`maker=100`, `taker=10`).
4. Operator calls `matchOrders(conditionId=B, ...)`.
5. Result: pooled settlement succeeds and `100 YES_B` becomes stranded in exchange.

Recommendation: Enforce market coherence per non-COMPLEMENTARY match on-chain. In v1, this was enforced through the on-chain registry via `validateComplement()`. To retain v2 gas savings, consider a design where each order includes signed `conditionId` and `complementTokenId` (or equivalent complement descriptor), and every maker order must satisfy the taker order's condition/complement relation at match time. This keeps validation lightweight, but still assumes off-chain infrastructure supplies correct market metadata before users sign.

POL-EX-5

Overflow in `calculateTakingAmount()` Can Produce Incorrect Settlement Amounts

Informational · Fixed

> **Update**
> Fixed in commit `ed9cad1aff2160dc995d61dcb54856501ee89842`.

Description: The `CalculatorHelper.calculateTakingAmount()` function uses inline assembly to compute `(makingAmount * takerAmount) / makerAmount`. The intermediate product `makingAmount * takerAmount` can silently overflow and wrap to a smaller value when it exceeds `type(uint256).max`. Off-chain validation may restrict the range of order parameters such that overflow does not occur in practice, but there is no on-chain logic that enforces or validates this.

The affected `takingAmount` is returned by `_performOrderChecks()` and is used inside both COMPLEMENTARY and non-COMPLEMENTARY paths to determine transfer amounts.

The overflow requires `order.makerAmount * order.takerAmount >= 2^256`. Since both amounts are price parameters in the signed order (not a transferred balance), it is not constrained by real token supply.

No scenario was identified in which this overflow could be used to systematically extract value from the opposing side of a match, if strict off-chain overflow checks are always applied to order and fill amounts.

Recommendation: Consider implementing either of the following overflow checks:

1. Assembly with explicit overflow check: Before multiplying in assembly, check that the product cannot overflow (e.g. require `makingAmount == 0 || takerAmount <= type(uint256).max / makingAmount`). For example, in assembly: `if gt(takerAmount, div(not(0), makingAmount)) { if makingAmount { store error and revert } }`.
2. Solidity checked arithmetic: Replace the assembly in `calculateTakingAmount()` with standard Solidity syntax and rely on its overflow checks: `return (makingAmount * takerAmount) / makerAmount`.

Auditor Suggestions

S1 Optimizing Gas Costs by Minimizing Transfers

Fixed

> **Update**
> Fixed in commit `5d341cec6902bb80bdcc08251312104ac11e4a22`.

File(s) affected: `src/exchange/mixins/Trading.sol`

Description: The settlement logic performs more ERC-20 / ERC-1155 transfer() and transferFrom() calls than necessary. The transfer counts below assume non-zero fee inputs where fee-transfer paths are present. Each redundant call adds gas overhead that scales linearly with the number of makers. The table below shows the current transfer count per flow (N = number of makers), the optimal count, and the number of wasted transfers. Note that these optimizations are proposed as minimal, local changes to existing settlement flows, without architecture-level redesign.

Flow	Current	Optimal	Wasted
COMPLEMENTARY (taker BUY)	2N + 2	2N + 1	1
COMPLEMENTARY (taker SELL)	3N + 2	2N + 2	N
Buy / MINT	3N + 3	2N + 3	N

Flow	Current	Optimal	Wasted
Sell / MERGE	2N + 4	2N + 4	0

All of the eliminated transfers touch storage slots that are already warm (e.g. fee receiver and participant balances used earlier in the same call). Savings therefore reflect warm SSTORE costs (100 gas) rather than cold (2200 gas), so the per-transfer gain is smaller than it would be for first writes to cold slots.

The current per-flow transfer breakdowns and the optimizations to reach the optimal counts are described below. The optimizations reuse two patterns referenced throughout:

- **Pattern A** (combine fee transfers): when two or more fee transfers share the same sender and the same recipient, replace them with a single transfer of the sum.
- **Pattern B** (pool BUY fees through the exchange): instead of each BUY participant calling `_chargeFee()` (which performs an individual `transferFrom` to the fee receiver), include the fee in the collateral transfer to the exchange (`fillAmount + feeAmount`), accumulate all fees, and transfer once to the fee receiver after the loop.

COMPLEMENTARY (taker BUY): 2N + 2 → 2N + 1.

- Current transfers:
  - CTF: maker → taker (N),
  - collateral: taker → maker net of fee (N),
  - batched maker SELL fees: taker → feeReceiver (1)
  - taker BUY fee: taker → feeReceiver (1)

The last two transfers are both collateral from the taker to the fee receiver.

- Improved transfers: apply Pattern A to combine the maker fee batch and taker fee into one `_transfer(taker, getFeeReceiver(), 0, totalMakerFees + takerFeeAmount)`, removing one collateral transfer.

COMPLEMENTARY (taker SELL): 3N + 2 → 2N + 2.

- Current transfers:
  - CTF: taker → maker (N)
  - collateral: maker → exchange (N)
  - maker BUY fee: maker → feeReceiver (N)
  - taker SELL fee: exchange → feeReceiver (1)
  - collateral: exchange → taker (1)
- Improved transfers: apply Pattern B so each maker sends `fillAmount + feeAmount` to the exchange in a single transfer, then batch all maker BUY fees in one `exchange → feeReceiver` transfer. Apply Pattern A on top to combine that maker fee batch with the taker SELL fee into one `exchange → feeReceiver` transfer. This removes N individual maker fee transfers while keeping the taker fee semantics unchanged.

Buy / MINT: 3N + 3 → 2N + 3.

- Current transfers:
  - collateral: taker → exchange (1)
  - collateral: maker → exchange (N)
  - CTF: exchange → maker proceeds (N)
  - maker BUY fee: maker → feeReceiver (N)
  - CTF: exchange → taker (1)
  - taker BUY fee: taker → feeReceiver (1)
- Improved transfers: apply Pattern B so each BUY maker sends `fillAmount + feeAmount` and the BUY taker sends `takerFillAmount + takerFeeAmount` as single collateral transfers to the exchange, and accumulate all BUY fees inside the exchange. Apply Pattern A to pay all maker and taker BUY fees in one `exchange → feeReceiver` transfer. Together, N + 1 individual fee transfers collapse into one batch transfer, saving N transfers.

Sell / MERGE: 2N + 4 (no safe reduction).

- Unchanged. The maker SELL fees must be transferred out of the exchange before the surplus / refund logic runs, while the taker SELL fee is derived only after that check.

Note that several of these optimizations merge what were previously separate "maker settlement" and "taker settlement" steps. For example, in the COMPLEMENTARY taker BUY optimization the taker pays their own fee and the makers' fees in one shot. In Pattern B flows, maker fees no longer flow directly from each maker to the fee receiver but are pooled through the exchange and batched with the taker fee. These changes alter intermediate token flows but preserve the same net economic outcome per party.

Recommendation: Consider implementing patterns A and B across all settlement paths as described above. In particular, the MINT / Buy path and any flows with many BUY makers benefit the most from pooling fees through the exchange and batching them, while the SELL / MERGE path is constrained by the surplus accounting and cannot safely batch taker and maker fee transfers without deeper structural changes.

S2 Improve Validations

Fixed

Update

Fixed in commit `bee78eed9188a7d25f8edffee1d9973b95ca7f7d`.

File(s) affected: `src/exchange/CTFExchange.sol`, `src/exchange/mixins/UserPausable.sol`

Description: Consider adding explicit input validations so invalid arguments fail with clear errors. The following is a non-exhaustive list of suggested validations:

1. In `CTFExchange.matchOrders()` require that `makerOrders`, `makerFillAmounts`, and `makerFeeAmounts` have the same length and have length at least 1.
2. In `CTFExchange.matchOrders()`, require that `tokenId` in the taker and maker orders is not `0`, because `0` is the sentinel value used for collateral tokens and allowing it can lead to collateral-for-collateral trades or wrong asset distribution in MINT/MERGE paths.
3. In `UserPausable.pauseUser()` require that an already paused user cannot call `pauseUser()` again. Calling it again temporarily unpauses them for `userPauseBlockInterval` blocks which is counterintuitive UX.
4. In `UserPausable.setUserPauseBlockInterval()` require an upper bound on the interval (e.g. `blockInterval <= MAX_PAUSE_INTERVAL`). An upper bound ensures that when users call `pauseUser()`, they do not have to wait an unreasonably long time before their account is actually paused.

Recommendation: Consider implementing the listed validation checks.

S3 Miscellaneous Improvements

Fixed

Update

Fixed in commit `b54b409d10129689a045547a7bd831e3793e53f0`.

File(s) affected: `src/exchange/libraries/CalculatorHelper.sol`, `src/exchange/mixins/Auth.sol`

Description: The following improvements would reduce bytecode size, audit surface, or maintenance burden:

1. Dead code in `CalculatorHelper`: The functions `_calculatePrice()`, `isCrossing()`, and `_isCrossing()` are not called anywhere and should be removed.
2. Require at least one admin: `Auth.removeAdmin()` does not prevent the last admin from being removed. If all admins are removed, admin functions become permanently inaccessible. Maintain an `adminCount` and require `adminCount > 1` before clearing an admin.
3. Redundant Code in `Trading`: The `_chargeFee()` function includes a check to ensure `fee > 0`, however, in each location the `_chargeFee()` function is called this check has already been made. The duplicate check can be removed.

Recommendation: Consider implementing the improvements listed above.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Polymarket/ctf-exchange-v2`

- `640...c63 ./src/adapters/CtfCollateralAdapter.sol`
- `f5f...334 ./src/adapters/NegRiskCtfCollateralAdapter.sol`
- `33a...4a3 ./src/adapters/libraries/CTFHelpers.sol`
- `285...23f ./src/adapters/libraries/CTHelpers.sol`

- `a0f...527 ./src/collateral/CollateralOfframp.sol`
- `9c7...68a ./src/collateral/CollateralOnramp.sol`
- `d0a...3ce ./src/collateral/CollateralToken.sol`
- `4be...a82 ./src/collateral/PermissionedRamp.sol`
- `d0d...121 ./src/collateral/abstract/CollateralErrors.sol`
- `609...864 ./src/collateral/abstract/Pausable.sol`
- `5fb...3a1 ./src/exchange/CTFExchange.sol`
- `ff1...888 ./src/exchange/libraries/CalculatorHelper.sol`
- `dd6...e22 ./src/exchange/libraries/Create2Lib.sol`
- `332...5f8 ./src/exchange/libraries/PolyProxyLib.sol`
- `c95...b09 ./src/exchange/libraries/PolySafeLib.sol`
- `8b2...292 ./src/exchange/libraries/Structs.sol`
- `778...88d ./src/exchange/libraries/TransferHelper.sol`
- `ca3...f90 ./src/exchange/mixins/AssetOperations.sol`
- `5ca...016 ./src/exchange/mixins/Assets.sol`
- `104...b21 ./src/exchange/mixins/Auth.sol`
- `bd6...fb7 ./src/exchange/mixins/ERC1155TokenReceiver.sol`
- `ebb...081 ./src/exchange/mixins/Events.sol`
- `637...ee9 ./src/exchange/mixins/Fees.sol`
- `68f...223 ./src/exchange/mixins/Hashing.sol`
- `5e5...4fd ./src/exchange/mixins/Pausable.sol`
- `982...c3f ./src/exchange/mixins/PolyFactoryHelper.sol`
- `000...172 ./src/exchange/mixins/Signatures.sol`
- `67d...42c ./src/exchange/mixins/Trading.sol`
- `830...d2f ./src/exchange/mixins/UserPausable.sol`

Test Suite Results

Test data output was obtained with `forge test`. All 22 test suites with 227 tests in total executed successfully. The client should consider adding targeted regression tests for each finding to verify that fixes are implemented correctly and remain effective in future updates.

Update Fix-Review: The test suite was expanded slightly and now contains 265 tests.

```
Ran 1 test for src/test/CollateralSetup.t.sol:CollateralSetUp_Test
[PASS] test_setup() (gas: 37221)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 381.75ms (22.66ms CPU time)

Ran 5 tests for src/test/libraries/PolyProxyLib.t.sol:PolyProxyLibTest
[PASS] test_PolyProxyLib_ProxyDeploymentMatchesPredictionAndDelegates() (gas: 90120)
[PASS] test_PolyProxyLib_canDeployMinimalImplementation() (gas: 204388)
[PASS] test_PolyProxyLib_computeCreationCodeEmbedsDependencies() (gas: 54809)
[PASS] test_PolyProxyLib_getProxyWalletAddressMatchesManualDerivation() (gas: 13735)
[PASS] test_PolyProxyLib_vmGetCodeDoesNotExposeProxyCreationCode() (gas: 12284)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 398.25ms (103.81ms CPU time)

Ran 4 tests for src/test/libraries/PolySafeLib.t.sol:PolySafeLibTest
[PASS] test_PolySafeLib_SafeDeploymentMatchesPredictionAndPersistsSingleton() (gas: 98714)
[PASS] test_PolySafeLib_computeBytecodeHashMatchesReference() (gas: 56985)
[PASS] test_PolySafeLib_getContractBytecodeReflectsImplementationAddress() (gas: 66909)
[PASS] test_PolySafeLib_getSafeWalletAddressMatchesManualDerivation() (gas: 14468)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.52ms (6.22ms CPU time)

Ran 6 tests for src/test/CollateralOfframp.t.sol:CollateralOfframpTest
[PASS] test_CollateralOfframp_unwrapUSDC() (gas: 235736)
[PASS] test_CollateralOfframp_unwrapUSDCe() (gas: 235824)
[PASS] test_Pausable_unpause() (gas: 259557)
[PASS] test_revert_CollateralOfframp_unwrapUSDC_paused() (gas: 206114)
[PASS] test_revert_CollateralOfframp_unwrapUSDCe_paused() (gas: 206089)
[PASS] test_revert_Pausable_pause_unauthorized() (gas: 15086)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 17.74ms (17.04ms CPU time)
```

```
Ran 6 tests for src/test/CollateralOnramp.t.sol:CollateralOnrampTest
[PASS] test_CollateralOnramp_wrapUSDC() (gas: 161677)
[PASS] test_CollateralOnramp_wrapUSDCe() (gas: 161729)
[PASS] test_Pausable_unpause() (gas: 187618)
[PASS] test_revert_CollateralOnramp_wrapUSDC_paused() (gas: 117330)
[PASS] test_revert_CollateralOnramp_wrapUSDCe_paused() (gas: 117341)
[PASS] test_revert_Pausable_pause_unauthorized() (gas: 15130)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.13ms (503.71µs CPU time)

Ran 15 tests for src/test/PermissionedRamp.t.sol:PermissionedRampTest
[PASS] test_PermissionedRamp_addWitness() (gas: 207416)
[PASS] test_PermissionedRamp_removeWitness() (gas: 133749)
[PASS] test_PermissionedRamp_unwrapUSDC() (gas: 265679)
[PASS] test_PermissionedRamp_unwrapUSDCe() (gas: 265691)
[PASS] test_PermissionedRamp_wrapUSDC() (gas: 190433)
[PASS] test_PermissionedRamp_wrapUSDCe() (gas: 190471)
[PASS] test_PermissionedRamp_wrap_incrementsNonce() (gas: 224495)
[PASS] test_revert_PermissionedRamp_unwrap_expiredDeadline() (gas: 211977)
[PASS] test_revert_PermissionedRamp_unwrap_invalidWitness() (gas: 216828)
[PASS] test_revert_PermissionedRamp_unwrap_paused() (gas: 238045)
[PASS] test_revert_PermissionedRamp_wrap_expiredDeadline() (gas: 98048)
[PASS] test_revert_PermissionedRamp_wrap_invalidNonce() (gas: 120315)
[PASS] test_revert_PermissionedRamp_wrap_invalidWitness() (gas: 124123)
[PASS] test_revert_PermissionedRamp_wrap_paused() (gas: 124952)
[PASS] test_revert_PermissionedRamp_wrap_replaySignature() (gas: 218589)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 515.29ms (277.79ms CPU time)

Ran 8 tests for src/test/MatchOrdersCtfCollateralAdapter.t.sol:MatchOrdersCtfCollateralAdapterTest
[PASS] test_MatchOrdersCtfCollateralAdapter_Complementary() (gas: 495493)
[PASS] test_MatchOrdersCtfCollateralAdapter_Complementary_Fees() (gas: 524195)
[PASS] test_MatchOrdersCtfCollateralAdapter_Merge() (gas: 695292)
[PASS] test_MatchOrdersCtfCollateralAdapter_Merge_Fees() (gas: 726528)
[PASS] test_MatchOrdersCtfCollateralAdapter_Merge_Reverts_WhenAdapterNotApproved() (gas: 494799)
[PASS] test_MatchOrdersCtfCollateralAdapter_Mint() (gas: 664485)
[PASS] test_MatchOrdersCtfCollateralAdapter_Mint_Fees() (gas: 692876)
[PASS] test_MatchOrdersCtfCollateralAdapter_Mint_Reverts_WhenAdapterUsdceMismatch() (gas: 7682933)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 526.22ms (39.52ms CPU time)

Ran 8 tests for
src/test/MatchOrdersNegRiskCtfCollateralAdapter.t.sol:MatchOrdersNegRiskCtfCollateralAdapterTest
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Complementary() (gas: 643238)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Complementary_Fees() (gas: 671957)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Merge() (gas: 1036838)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Merge_Fees() (gas: 1068055)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Merge_Reverts_WhenAdapterNotApproved() (gas: 736289)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Mint() (gas: 810652)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Mint_Fees() (gas: 838982)
[PASS] test_MatchOrdersNegRiskCtfCollateralAdapter_Mint_Reverts_WhenAdapterUsdceMismatch() (gas: 8267463)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 531.97ms (17.78ms CPU time)

Ran 6 tests for src/test/ERC1271Signature.t.sol:ERC1271SignatureTest
[PASS] test_ERC1271Signature_validate1271Signature() (gas: 33535)
[PASS] test_ERC1271Signature_validate1271Signature_revert_incorrectSigner() (gas: 30655)
[PASS] test_ERC1271Signature_validate1271Signature_revert_invalidContract() (gas: 24812)
[PASS] test_ERC1271Signature_validate1271Signature_revert_invalidSignerMaker() (gas: 23552)
[PASS] test_ERC1271Signature_validate1271Signature_revert_nonContract() (gas: 24086)
[PASS] test_ERC1271Signature_validate1271Signature_revert_sigType() (gas: 24738)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 9.07ms (1.95ms CPU time)

Ran 36 tests for src/test/CTFExchange.t.sol:CTFExchangeTest
[PASS] test_CTFExchange_Auth() (gas: 75557)
[PASS] test_CTFExchange_Auth_NotAdmin() (gas: 10629)
[PASS] test_CTFExchange_Auth_RemoveAdmin() (gas: 62242)
[PASS] test_CTFExchange_Auth_RemoveAdminWithMultiple() (gas: 36026)
[PASS] test_CTFExchange_Auth_RenounceOperator() (gas: 14837)
[PASS] test_CTFExchange_Auth_revert_AddExistingAdmin() (gas: 13394)
[PASS] test_CTFExchange_Auth_revert_AddExistingOperator() (gas: 15435)
[PASS] test_CTFExchange_Auth_revert_RemoveLastAdmin() (gas: 15538)
[PASS] test_CTFExchange_Auth_revert_RemoveNonAdmin() (gas: 17370)
[PASS] test_CTFExchange_Auth_revert_RemoveNonOperator() (gas: 17455)
[PASS] test_CTFExchange_DefaultMaxFeeRate() (gas: 7648)
```

```
[PASS] test_CTFExchange_Pause() (gas: 666628)
[PASS] test_CTFExchange_SetFeeReceiver() (gas: 21538)
[PASS] test_CTFExchange_SetMaxFeeRate() (gas: 21153)
[PASS] test_CTFExchange_SetMaxFeeRate_revert_ExceedsCeiling() (gas: 13078)
[PASS] test_CTFExchange_SetMaxFeeRate_revert_NotAdmin() (gas: 13018)
[PASS] test_CTFExchange_SetUserPauseBlockInterval() (gas: 22653)
[PASS] test_CTFExchange_SetUserPauseBlockInterval_MaxValue() (gas: 19945)
[PASS] test_CTFExchange_SetUserPauseBlockInterval_revert_ExceedsMax() (gas: 13060)
[PASS] test_CTFExchange_UserPausable_rePauseAfterUnpause() (gas: 45204)
[PASS] test_CTFExchange_UserPausable_revert_UserAlreadyPaused() (gas: 37954)
[PASS] test_CTFExchange_UserPaused_ExactBoundaryBlock() (gas: 58000)
[PASS] test_CTFExchange_ValidateFee() (gas: 9090)
[PASS] test_CTFExchange_ValidateFee_ZeroRate() (gas: 15863)
[PASS] test_CTFExchange_ValidateFee_revert_FeeExceedsMaxRate() (gas: 10771)
[PASS] test_CTFExchange_ValidateOrder() (gas: 27770)
[PASS] test_CTFExchange_ValidateOrder_UserPaused() (gas: 63598)
[PASS] test_CTFExchange_ValidateOrder_revert_DuplicateOrder() (gas: 650572)
[PASS] test_CTFExchange_ValidateOrder_revert_InvalidSig() (gas: 27245)
[PASS] test_CTFExchange_ValidateOrder_revert_InvalidSigLength() (gas: 19993)
[PASS] test_CTFExchange_ValidateOrder_revert_InvalidSignerMaker() (gas: 28352)
[PASS] test_CTFExchange_hashOrder() (gas: 12168)
[PASS] test_CTFExchange_matchOrders_revert_MismatchedArrayLengths() (gas: 40955)
[PASS] test_CTFExchange_matchOrders_revert_NoMakerOrders() (gas: 30540)
[PASS] test_CTFExchange_setup() (gas: 21859)
[PASS] test_CTFExchange_supportsInterface() (gas: 7661)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 606.42ms (109.52ms CPU time)

Ran 16 tests for src/test/GasSnapshots.t.sol:GasSnapshots
[PASS] test_GasSnapshots_combo_complementary_merge_10makers() (gas: 1302505)
[PASS] test_GasSnapshots_combo_complementary_merge_20makers() (gas: 1723583)
[PASS] test_GasSnapshots_combo_complementary_mint_10makers() (gas: 1317796)
[PASS] test_GasSnapshots_combo_complementary_mint_20makers() (gas: 1741000)
[PASS] test_GasSnapshots_complementary_10makers() (gas: 1010408)
[PASS] test_GasSnapshots_complementary_1maker() (gas: 649064)
[PASS] test_GasSnapshots_complementary_20makers() (gas: 1514171)
[PASS] test_GasSnapshots_complementary_5makers() (gas: 809456)
[PASS] test_GasSnapshots_merge_10makers() (gas: 1114516)
[PASS] test_GasSnapshots_merge_1maker() (gas: 732330)
[PASS] test_GasSnapshots_merge_20makers() (gas: 1614478)
[PASS] test_GasSnapshots_merge_5makers() (gas: 901854)
[PASS] test_GasSnapshots_mint_10makers() (gas: 1008146)
[PASS] test_GasSnapshots_mint_1maker() (gas: 636825)
[PASS] test_GasSnapshots_mint_20makers() (gas: 1486427)
[PASS] test_GasSnapshots_mint_5makers() (gas: 801548)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 94.65ms (221.39ms CPU time)

Ran 13 tests for src/test/GasSnapshotsCtfCollateralAdapter.t.sol:GasSnapshotsCtfCollateralAdapter
[PASS] test_GasSnapshotsCtfCollateralAdapter_combo_complementary_merge_10makers() (gas: 1196400)
[PASS] test_GasSnapshotsCtfCollateralAdapter_combo_complementary_merge_20makers() (gas: 1691318)
[PASS] test_GasSnapshotsCtfCollateralAdapter_combo_complementary_mint_10makers() (gas: 1202572)
[PASS] test_GasSnapshotsCtfCollateralAdapter_combo_complementary_mint_20makers() (gas: 1639150)
[PASS] test_GasSnapshotsCtfCollateralAdapter_complementary_10makers() (gas: 915098)
[PASS] test_GasSnapshotsCtfCollateralAdapter_complementary_1maker() (gas: 492126)
[PASS] test_GasSnapshotsCtfCollateralAdapter_complementary_5makers() (gas: 662698)
[PASS] test_GasSnapshotsCtfCollateralAdapter_merge_10makers() (gas: 1109822)
[PASS] test_GasSnapshotsCtfCollateralAdapter_merge_1maker() (gas: 690353)
[PASS] test_GasSnapshotsCtfCollateralAdapter_merge_5makers() (gas: 860856)
[PASS] test_GasSnapshotsCtfCollateralAdapter_mint_10makers() (gas: 1032864)
[PASS] test_GasSnapshotsCtfCollateralAdapter_mint_1maker() (gas: 659634)
[PASS] test_GasSnapshotsCtfCollateralAdapter_mint_5makers() (gas: 825204)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 62.70ms (127.38ms CPU time)

Ran 10 tests for src/test/BalanceDeltas.t.sol:BalanceDeltasTest
[PASS] test_BalanceDeltas_Complementary_TakerSell_WithTakerFee() (gas: 709312)
[PASS] test_BalanceDeltas_MultiMaker_Complementary_TakerBuy() (gas: 903792)
[PASS] test_BalanceDeltas_MultiMaker_Complementary_TakerSell() (gas: 932404)
[PASS] test_BalanceDeltas_MultiMaker_Complementary_TakerSell_WithFees() (gas: 959186)
[PASS] test_BalanceDeltas_MultiMaker_Complementary_WithFees() (gas: 931006)
[PASS] test_BalanceDeltas_MultiMaker_Merge() (gas: 967027)
[PASS] test_BalanceDeltas_MultiMaker_Merge_WithFees() (gas: 1000768)
[PASS] test_BalanceDeltas_MultiMaker_Mint() (gas: 905998)
[PASS] test_BalanceDeltas_MultiMaker_Mint_WithFees() (gas: 928933)
```

```
[PASS] test_BalanceDeltas_VariedFillAmounts_DifferentPrices() (gas: 899439)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 655.08ms (74.58ms CPU time)

Ran 13 tests for src/test/Preapproved.t.sol:PreapprovedTest
[PASS] test_matchOrders_emptySignature_invalidatedPreapproval_reverts() (gas: 30765)
[PASS] test_matchOrders_emptySignature_notPreapproved_reverts() (gas: 34754)
[PASS] test_matchOrders_emptySignature_preapproved_complementary() (gas: 113113)
[PASS] test_matchOrders_invalidSignature_notPreapproved_reverts() (gas: 36268)
[PASS] test_matchOrders_invalidatedPreapproval_reverts() (gas: 37847)
[PASS] test_matchOrders_preapproved1271_signerInvalidated() (gas: 131855)
[PASS] test_matchOrders_preapprovedMaker_complementary() (gas: 115944)
[PASS] test_matchOrders_preapprovedTaker_complementary() (gas: 115962)
[PASS] test_matchOrders_preapproved_partialFill() (gas: 77672)
[PASS] test_matchOrders_preapproved_respectsFilledStatus() (gas: 30713)
[PASS] test_matchOrders_preapproved_respectsUserPause() (gas: 31050)
[PASS] test_preapproveOrder_revert_NotOperator() (gas: 22095)
[PASS] test_preapproveOrder_revert_invalidSignature() (gas: 29611)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 660.94ms (91.25ms CPU time)

Ran 9 tests for src/test/GasSnapshotsSafe.t.sol:GasSnapshotsSafe
[PASS] test_GasSnapshotsSafe_complementary_10makers() (gas: 1100722)
[PASS] test_GasSnapshotsSafe_complementary_1maker() (gas: 660773)
[PASS] test_GasSnapshotsSafe_complementary_5makers() (gas: 841164)
[PASS] test_GasSnapshotsSafe_merge_10makers() (gas: 1159518)
[PASS] test_GasSnapshotsSafe_merge_1maker() (gas: 744400)
[PASS] test_GasSnapshotsSafe_merge_5makers() (gas: 928519)
[PASS] test_GasSnapshotsSafe_mint_10makers() (gas: 1077550)
[PASS] test_GasSnapshotsSafe_mint_1maker() (gas: 648175)
[PASS] test_GasSnapshotsSafe_mint_5makers() (gas: 832912)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 661.22ms (146.08ms CPU time)

Ran 13 tests for
src/test/GasSnapshotsNegRiskCtfCollateralAdapter.t.sol:GasSnapshotsNegRiskCtfCollateralAdapter
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_combo_complementary_merge_10makers() (gas: 1537892)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_combo_complementary_merge_20makers() (gas: 1960992)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_combo_complementary_mint_10makers() (gas: 1447820)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_combo_complementary_mint_20makers() (gas: 1870768)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_complementary_10makers() (gas: 1002981)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_complementary_1maker() (gas: 639869)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_complementary_5makers() (gas: 801024)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_merge_10makers() (gas: 1416200)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_merge_1maker() (gas: 1031905)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_merge_5makers() (gas: 1202395)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_mint_10makers() (gas: 1179020)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_mint_1maker() (gas: 805753)
[PASS] test_GasSnapshotsNegRiskCtfCollateralAdapter_mint_5makers() (gas: 971342)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 661.59ms (142.73ms CPU time)

Ran 9 tests for src/test/GasSnapshotsProxy.t.sol:GasSnapshotsProxy
[PASS] test_GasSnapshotsProxy_complementary_10makers() (gas: 1098921)
[PASS] test_GasSnapshotsProxy_complementary_1maker() (gas: 657898)
[PASS] test_GasSnapshotsProxy_complementary_5makers() (gas: 838924)
[PASS] test_GasSnapshotsProxy_merge_10makers() (gas: 1159236)
[PASS] test_GasSnapshotsProxy_merge_1maker() (gas: 741554)
[PASS] test_GasSnapshotsProxy_merge_5makers() (gas: 926777)
[PASS] test_GasSnapshotsProxy_mint_10makers() (gas: 1075727)
[PASS] test_GasSnapshotsProxy_mint_1maker() (gas: 645304)
[PASS] test_GasSnapshotsProxy_mint_5makers() (gas: 830706)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 661.78ms (193.98ms CPU time)

Ran 32 tests for src/test/CollateralToken.t.sol:CollateralTokenTest
[PASS] test_CollateralToken_addMinter() (gas: 44872)
[PASS] test_CollateralToken_addWrapper() (gas: 44895)
[PASS] test_CollateralToken_burn() (gas: 54213)
[PASS] test_CollateralToken_decimals() (gas: 10292)
[PASS] test_CollateralToken_immutables() (gas: 18988)
[PASS] test_CollateralToken_initialize() (gas: 14553)
[PASS] test_CollateralToken_mint() (gas: 69412)
[PASS] test_CollateralToken_name() (gas: 14025)
[PASS] test_CollateralToken_permit2NoInfiniteAllowance() (gas: 14881)
[PASS] test_CollateralToken_removeMinter() (gas: 35887)
[PASS] test_CollateralToken_removeWrapper() (gas: 35905)
```

```
[PASS] test_CollateralToken_symbol() (gas: 14100)
[PASS] test_CollateralToken_unwrapUSDC() (gas: 236256)
[PASS] test_CollateralToken_unwrapUSDC_noCallback() (gas: 211844)
[PASS] test_CollateralToken_unwrapUSDCe() (gas: 236328)
[PASS] test_CollateralToken_upgradeToAndCall() (gas: 1478795)
[PASS] test_CollateralToken_wrapUSDC() (gas: 167292)
[PASS] test_CollateralToken_wrapUSDC_noCallback() (gas: 132416)
[PASS] test_CollateralToken_wrapUSDCe() (gas: 167318)
[PASS] test_CollateralToken_wrapUSDCe_noCallback() (gas: 132430)
[PASS] test_revert_CollateralToken_addMinter_unauthorized() (gas: 17825)
[PASS] test_revert_CollateralToken_addWrapper_unauthorized() (gas: 17871)
[PASS] test_revert_CollateralToken_burn_unauthorized() (gas: 19845)
[PASS] test_revert_CollateralToken_initialize_alreadyInitialized() (gas: 20453)
[PASS] test_revert_CollateralToken_mint_unauthorized() (gas: 20054)
[PASS] test_revert_CollateralToken_removeMinter_unauthorized() (gas: 17872)
[PASS] test_revert_CollateralToken_removeWrapper_unauthorized() (gas: 17848)
[PASS] test_revert_CollateralToken_unwrapInvalidAsset(address) (runs: 256, μ: 27502, ~: 27502)
[PASS] test_revert_CollateralToken_unwrap_unauthorized() (gas: 24672)
[PASS] test_revert_CollateralToken_upgradeToAndCall_unauthorized() (gas: 1474492)
[PASS] test_revert_CollateralToken_wrapInvalidAsset(address) (runs: 256, μ: 27500, ~: 27500)
[PASS] test_revert_CollateralToken_wrap_unauthorized() (gas: 24715)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 255.90ms (351.47ms CPU time)

Ran 33 tests for src/test/MatchOrders.t.sol:MatchOrdersTest
[PASS] test_MatchOrders_Complementary() (gas: 120341)
[PASS] test_MatchOrders_Complementary_ConsumesActualTakerFill_WhenPriceImproves() (gas: 658764)
[PASS] test_MatchOrders_Complementary_Fees() (gas: 145864)
[PASS] test_MatchOrders_Complementary_Fees_Surplus() (gas: 165625)
[PASS] test_MatchOrders_Complementary_NoExchangeBalance_TakerBuy() (gas: 120392)
[PASS] test_MatchOrders_Complementary_NoExchangeBalance_TakerSell() (gas: 140522)
[PASS] test_MatchOrders_Events_Complementary_WithFees() (gas: 692953)
[PASS] test_MatchOrders_Merge() (gas: 212001)
[PASS] test_MatchOrders_Merge_Fees() (gas: 239784)
[PASS] test_MatchOrders_Mint() (gas: 238384)
[PASS] test_MatchOrders_Mint_Fees() (gas: 263488)
[PASS] test_MatchOrders_Mint_Fees_TakerRefund_PooledFeesNotRefunded() (gas: 698276)
[PASS] test_MatchOrders_TakerRefund() (gas: 135833)
[PASS] test_MatchOrders_WithMaxFeeRate_Buy() (gas: 144137)
[PASS] test_MatchOrders_WithMaxFeeRate_Sell() (gas: 163946)
[PASS] test_MatchOrders_ZeroTakerAmount() (gas: 120339)
[PASS] test_MatchOrders_revert_ComplementaryFillExceedsTakerFill_Overspend() (gas: 624262)
[PASS] test_MatchOrders_revert_FeeExceedsMaxRate_Buy() (gas: 626461)
[PASS] test_MatchOrders_revert_FeeExceedsMaxRate_BuyWithNoMakersAndZeroFill() (gas: 217008)
[PASS] test_MatchOrders_revert_FeeExceedsMaxRate_Sell() (gas: 626454)
[PASS] test_MatchOrders_revert_FeeExceedsProceeds() (gas: 650398)
[PASS] test_MatchOrders_revert_InvalidFillAmount() (gas: 626323)
[PASS] test_MatchOrders_revert_InvalidTrade() (gas: 556271)
[PASS] test_MatchOrders_revert_MismatchedTokenIds_InvalidTokenId() (gas: 457095)
[PASS] test_MatchOrders_revert_MismatchedTokenIds_MintSameTokenId() (gas: 661937)
[PASS] test_MatchOrders_revert_MismatchedTokenIds_TokenIdZeroCollateralExtraction() (gas: 420160)
[PASS] test_MatchOrders_revert_NoMakerOrders_Buy() (gas: 216944)
[PASS] test_MatchOrders_revert_NoMakerOrders_Sell() (gas: 345692)
[PASS] test_MatchOrders_revert_NotCrossingBuyVsSell() (gas: 554877)
[PASS] test_MatchOrders_revert_NotCrossingBuys() (gas: 471820)
[PASS] test_MatchOrders_revert_NotCrossingSells() (gas: 591030)
[PASS] test_MatchOrders_revert_ZeroMakerAmount() (gas: 65075)
[PASS] test_MatchOrders_revert_ZeroMakerAmount_Taker() (gas: 367607)
Suite result: ok. 33 passed; 0 failed; 0 skipped; finished in 661.57ms (213.48ms CPU time)

Ran 2 tests for src/test/libraries/CalculatorHelper.t.sol:CalculatorHelperTest
[PASS] test_CalculatorHelper_FuzzCalculateTakingAmount(uint64,uint128,uint128) (runs: 256, μ: 3865, ~:
3865)
[PASS] test_CalculatorHelper_revert_CalculateTakingAmountOverflow() (gas: 3892)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 671.47ms (420.19ms CPU time)

Ran 8 tests for src/test/CtfCollateralAdapter.t.sol:CtfCollateralAdapterTest
[PASS] test_CtfCollateralAdapter_mergePositions() (gas: 653241)
[PASS] test_CtfCollateralAdapter_redeemPositions(bool) (runs: 256, μ: 766094, ~: 766077)
[PASS] test_CtfCollateralAdapter_splitPosition() (gas: 398505)
[PASS] test_CtfCollateralAdapter_unpause() (gas: 422590)
[PASS] test_revert_CtfCollateralAdapter_mergePositions_paused() (gas: 501520)
[PASS] test_revert_CtfCollateralAdapter_pause_unauthorized() (gas: 15088)
```

```
[PASS] test_revert_CtfCollateralAdapter_redeemPositions_paused() (gas: 489340)
[PASS] test_revert_CtfCollateralAdapter_splitPosition_paused() (gas: 209571)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 620.58ms (594.56ms CPU time)

Ran 12 tests for src/test/NegRiskCtfCollateralAdapter.t.sol:NegRiskCtfCollateralAdapterTest
[PASS] test_NegRiskCtfCollateralAdapter_convertPositions_oneNoToYes() (gas: 2384802)
[PASS] test_NegRiskCtfCollateralAdapter_convertPositions_threeNoToYes() (gas: 2274824)
[PASS] test_NegRiskCtfCollateralAdapter_convertPositions_twoNoToYes() (gas: 2362723)
[PASS] test_NegRiskCtfCollateralAdapter_convertPositions_withFees() (gas: 2403435)
[PASS] test_NegRiskCtfCollateralAdapter_convertPositions_zeroAmount() (gas: 1840510)
[PASS] test_NegRiskCtfCollateralAdapter_mergePositions() (gas: 1223080)
[PASS] test_NegRiskCtfCollateralAdapter_redeemPositions(bool) (runs: 256, μ: 1392000, ~: 1392135)
[PASS] test_NegRiskCtfCollateralAdapter_splitPosition() (gas: 887579)
[PASS] test_revert_NegRiskCtfCollateralAdapter_convertPositions_paused() (gas: 1794037)
[PASS] test_revert_NegRiskCtfCollateralAdapter_mergePositions_paused() (gas: 990880)
[PASS] test_revert_NegRiskCtfCollateralAdapter_redeemPositions_paused() (gas: 975087)
[PASS] test_revert_NegRiskCtfCollateralAdapter_splitPosition_paused() (gas: 640719)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 1.68s (1.24s CPU time)

Ran 22 test suites in 1.82s (10.34s CPU time): 265 tests passed, 0 failed, 0 skipped (265 total tests)
```

Code Coverage

The test suite has high branch coverage of 85.6% , meaning that most paths are well tested. Next to unit tests, the test suite contains fuzz tests and end-to-end tests.

Update Fix-Review: The test suite was expanded and now covers 89.04% of all branches.

File	% Lines	% Statements	% Branches	% Funcs
src/adapters/CtfCollateralAdapter.sol	91.49% (43/47)	95.56% (43/45)	100.00% (0/0)	80.00% (8/10)
src/adapters/NegRiskCtfCollateralAdapter.sol	100.00% (48/48)	100.00% (58/58)	100.00% (3/3)	100.00% (7/7)
src/adapters/libraries/CTFHelpers.sol	100.00% (11/11)	100.00% (10/10)	100.00% (0/0)	100.00% (2/2)
src/adapters/libraries/CTHelpers.sol	97.44% (343/352)	96.65% (346/358)	25.00% (2/8)	100.00% (4/4)
src/collateral/CollateralOfframp.sol	63.64% (7/11)	71.43% (5/7)	100.00% (0/0)	50.00% (2/4)
src/collateral/CollateralOnramp.sol	63.64% (7/11)	71.43% (5/7)	100.00% (0/0)	50.00% (2/4)
src/collateral/CollateralToken.sol	100.00% (42/42)	100.00% (26/26)	100.00% (4/4)	100.00% (16/16)
src/collateral/PermissionedRamp.sol	86.67% (26/30)	91.67% (22/24)	100.00% (6/6)	77.78% (7/9)
src/collateral/abstract/Pausable.sol	100.00% (8/8)	100.00% (5/5)	100.00% (2/2)	100.00% (3/3)
src/exchange/CTFExchange.sol	100.00% (20/20)	100.00% (13/13)	100.00% (4/4)	100.00% (8/8)
src/exchange/libraries/CalculatorHelper.sol	100.00% (2/2)	100.00% (2/2)	100.00% (0/0)	100.00% (1/1)

File	% Lines	% Statements	% Branches	% Funcs
src/exchange/libraries/Create2Lib.sol	100.00% (6/6)	100.00% (5/5)	100.00% (0/0)	100.00% (1/1)
src/exchange/libraries/PolyProxyLib.sol	100.00% (19/19)	100.00% (17/17)	100.00% (0/0)	100.00% (4/4)
src/exchange/libraries/PolySafeLib.sol	100.00% (24/24)	100.00% (22/22)	100.00% (0/0)	100.00% (4/4)
src/exchange/libraries/TransferHelper.sol	75.00% (6/8)	75.00% (3/4)	100.00% (0/0)	75.00% (3/4)
src/exchange/mixins/AssetOperations.sol	100.00% (24/24)	100.00% (27/27)	100.00% (4/4)	100.00% (7/7)
src/exchange/mixins/Assets.sol	73.33% (11/15)	100.00% (10/10)	100.00% (0/0)	20.00% (1/5)
src/exchange/mixins/Auth.sol	100.00% (34/34)	100.00% (24/24)	92.86% (13/14)	100.00% (10/10)
src/exchange/mixins/ERC1155TokenReceiver.sol	100.00% (7/7)	100.00% (6/6)	100.00% (0/0)	100.00% (3/3)
src/exchange/mixins/Events.sol	100.00% (24/24)	100.00% (21/21)	100.00% (0/0)	100.00% (3/3)
src/exchange/mixins/Fees.sol	100.00% (20/20)	100.00% (16/16)	100.00% (6/6)	100.00% (7/7)
src/exchange/mixins/Hashing.sol	100.00% (11/11)	100.00% (9/9)	100.00% (0/0)	100.00% (3/3)
src/exchange/mixins/Pausable.sol	100.00% (8/8)	100.00% (5/5)	100.00% (2/2)	100.00% (3/3)
src/exchange/mixins/PolyFactoryHelper.sol	47.37% (9/19)	73.33% (11/15)	100.00% (0/0)	14.29% (1/7)
src/exchange/mixins/Signatures.sol	100.00% (32/32)	100.00% (41/41)	100.00% (14/14)	100.00% (10/10)
src/exchange/mixins/Trading.sol	98.64% (218/221)	97.93% (236/241)	88.00% (66/75)	100.00% (22/22)
src/exchange/mixins/UserPausable.sol	100.00% (16/16)	100.00% (16/16)	100.00% (4/4)	100.00% (4/4)
src/scripts/ExchangeDeployment.s.sol	0.00% (0/10)	0.00% (0/10)	100.00% (0/0)	0.00% (0/2)
Total	95.00% (1026/1080)	96.17% (1004/1044)	89.04% (130/146)	87.43% (146/167)

Changelog

- 2026-03-11 - Initial report
- 2026-03-25 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PandcakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.